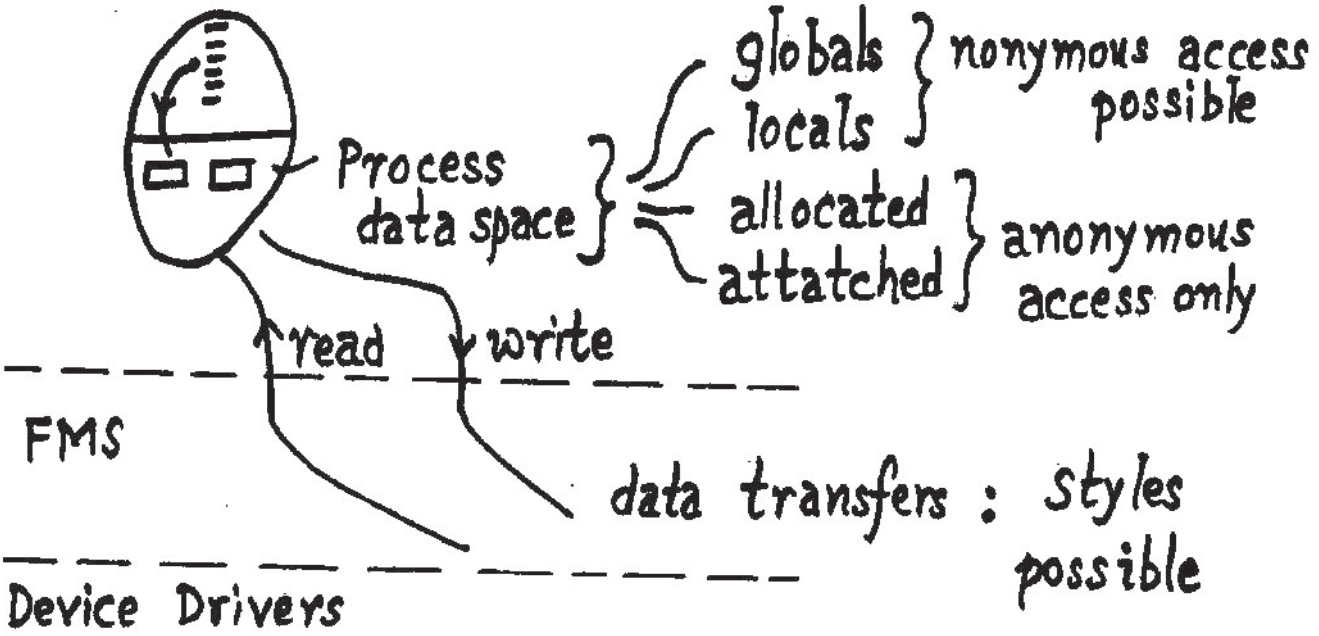
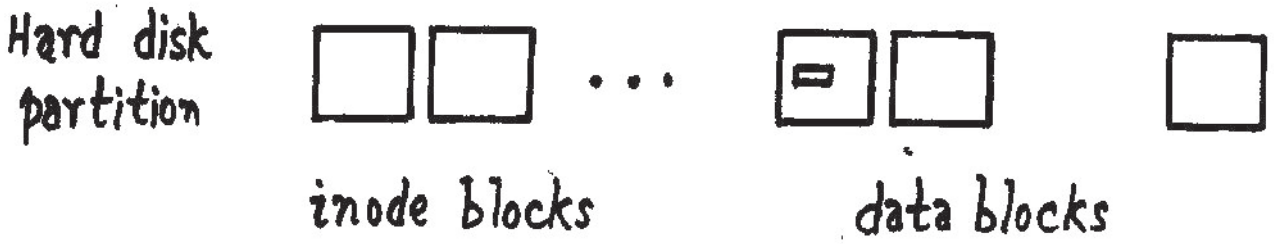
FMS IMPLEMENTATION

What are the constraints ?





A data box : in process data space

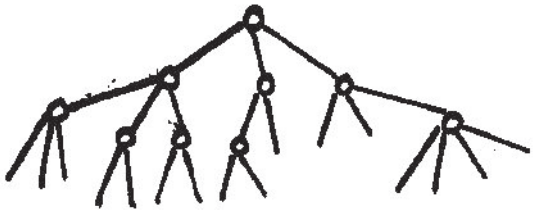
* a C datatype like double
* an array
* a structure

} almost always consecutive bytes

Data Blocks :

Not necessarily consecutive for a file

FMS implementation constraints



Tree of Files

Picture presented by FMS to processes

A File: FMS prepares data block number list

Analogy: Read a book

* read a sentance
* read a paragraph
* read a chapter

} parts may be in different pages

Again: Consecutive bytes transferred.

A File: [ø] [1] ... [N-1] a byte stream

consumption rate of bytes from a file

like: consumption of ground nuts from a plate of groundnuts

* one nut at a time
* many nuts at a time

FMS implementation constraints.

bottomline : access times

Example : 1 MB file

(a) one read of 1 MB

(b) 1 m reads of 1 byte each

Security issue

Process runs with permissions of euid. (Read geteuid(2))

Problem: enforcing security for each read.

Analogy: Visitor to a large corporation

Pathnames issue

Processes use pathnames

FMS: translates pathnames to block number list

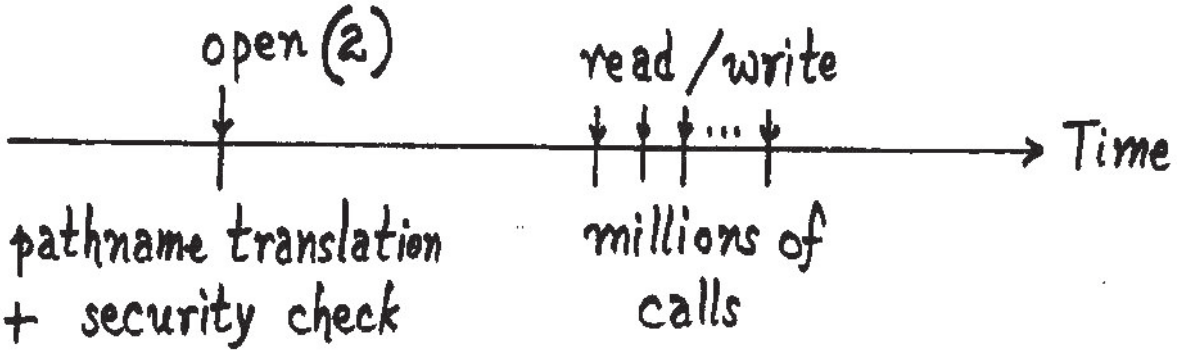
Problem: Doing conversion for each read.

Multi Access issue

* a process accessing several files
* several processes accessing a file
* a process accessing a file in multiple places

FMS implementation

Method: Security gate : only one security check for visits to all desired departments



Analogy: open a book & then do reading

open: specify pathname & list of accesses

Multi Access needs multiple data block number lists (OR inode informations)

analogy: read multiple books in library

⇒ provide a table

struct file (sys/file.h)
an open instantiation
: : :

File Table per process

FMS implementation concepts

Multi Access

analogy : trunk booking

Call identification

* not by source telephone number
* not by destination telephone number
* not by specifying both either

Use: token number / ticket no. (Ex: Z 123)
(Entry in a Register in Tel. Department)

FMS: Use Row # of file table to identify an open instantiation.

File Descriptor

* returned by open
* used as parameter by read/write

Question: No. of Rows in File Table ?

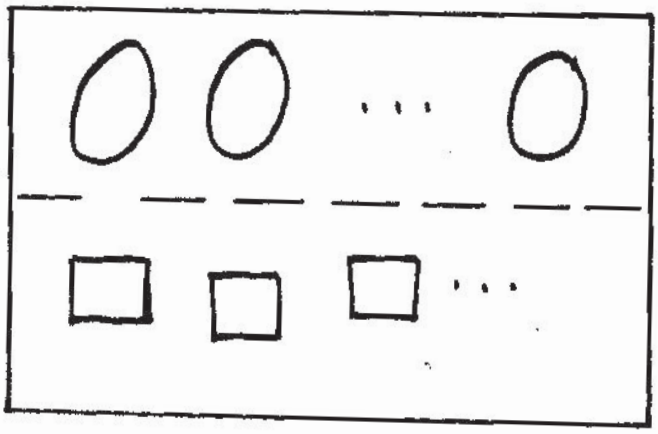
ANS: Depends on JOB MIX.

FMS implementation concepts

Job Mix

Student Exercise : 1/2 files

Database App. : 60 - 120 files opened



Core Memory : office space

Processes : Customers

File Tables : Secretarys' tables

Restriction: All File Tables are of same size.

NFILE: a tuneable parameter

* max. # of files a process can concurrently access

analogy: Library has a hierarchy.

* close(2): frees slot in file table

ACCESS TYPES all to be specified at open.

(a) read: Transfer from partition to process

(b) write: Transfer from process data space to partition data blocks

access types

(c) read & write : like idli & sambar

(d) analogy : a file is like an office file

* put sheet inside ⇒ write operation
* get a sheet ⇒ read
* open a file ⇒ create

creat (2) : system call creates a file

create if needed : is an access type in open.

* inode block allocated
* inode # + pathname component stored in a directory data block

⇒ Needs write permission in directory for euid of process

(e) scratch file : a fresh file needed

exclusive create : is an access type

(f) logfiles : multiple processes write
No rewrites. Use append.

(g) truncate : File size truncated to zero.

access types

(h) sync : synchronize

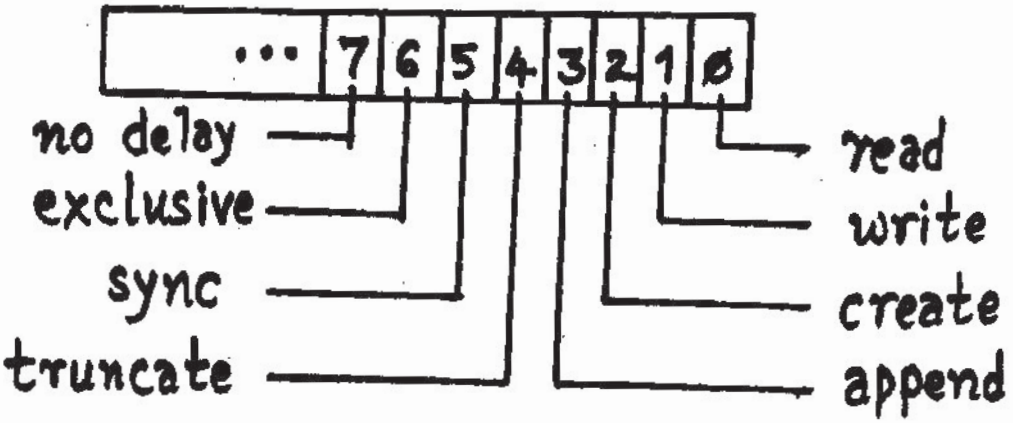
(i) no delay :

ACCESS SPECIFICATION

8 independant desires ⇒ 2^8 combinations

Ex: exclusive create with read
(like Jamoon & chutney)

a simple mechanism



Ex 1: Read & write : specify 3

Ex 2: Appended Synchronized writes into exclusively created file :
44, 106, 72, 78 ?

access specifications

Better method : use symbolic literals.

```
# define  READ   1
# define  WRITE  2
# define  CREATE 4
. . . . . .
```

open styles

(A) open (—, 78) /* appended sync ... */

(B) open (—, APPEND + SYNC + WRITE
+ EXCLUSIVE + CREATE);

documentation

good bad ugly

(A) is bad

(B) is ugly !
- Bit assignments may be wron
- # of # define's

fcntl.h : file control header file

Programs use : # include <fcntl.h>

a catch : CREATE : IPC facility also may need.

access specifications

fcntl.h uses names

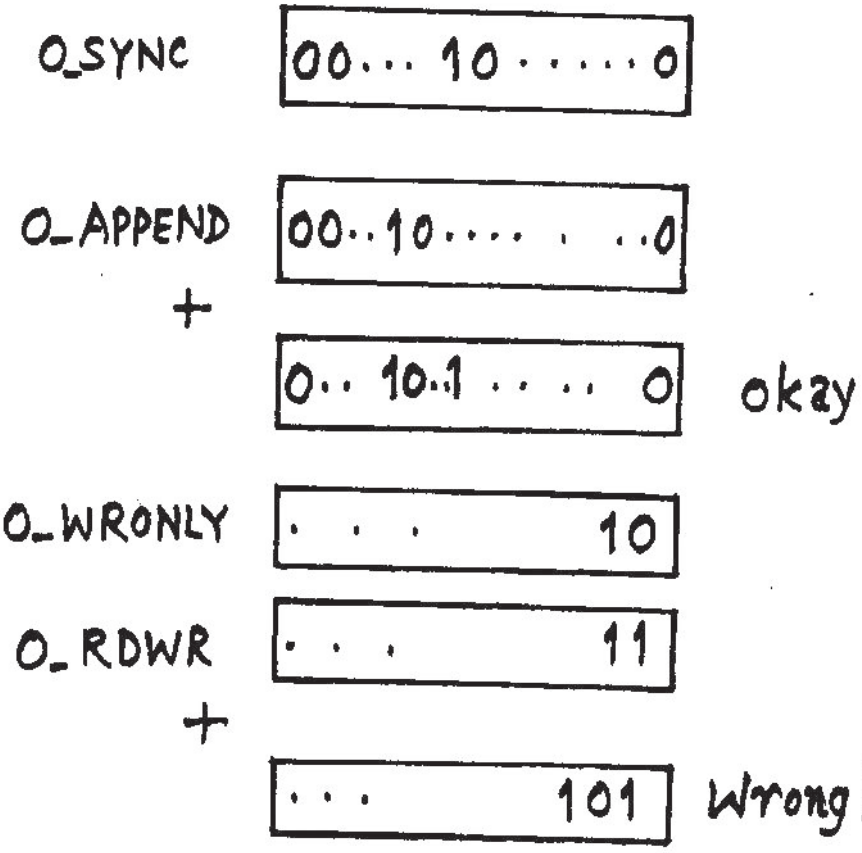
O_RDONLY, O_WRONLY, O_RDWR,

O_CREAT, O_TRUNC, O_SYNC,

O_APPEND, O_EXCL, O_NDELAY

Style (C): open(-, O_APPEND + O_SYNC + O_WRONLY + O_EXCL + O_CREAT)

Final Catch: '+' operator



(D) open(-, O_APPEND | O_SYNC | O_WRONLY | ...)

HEADER FILES contain

(a) OR-masks as symbolic literals

(b) other symbolic literals

(c) macro definitions

(d) structure definitions

(e) procedures :

Invokation – after definition

– before definition

template needed:

Ex: double dadd (double, double);

Ex: int open (char*, int);

– no definition

Ex: library procedure

All procedure templates / signatures provided.

Adds to : Documentation

(f) private members : of structures

Direct access ⇒ programs become non-portable.

These refer to layered architecture.

(g) plated throughs.